



Operations Infrastructure Roadmap

Kaakbay mo sa negosyo.

Internal Planning Document · Confidential

v1.0 · March 2026

Prepared by: Anton del Rosario

Companion Document: AKBai Complete Roadmap v8.0

CONFIDENTIAL — For Internal Planning Only

Why a Separate Operations Roadmap

The product roadmap covers what users see and interact with — the PWA, chat interface, dashboards, BIR features. The operations roadmap covers what Anton sees and manages: the internal infrastructure that keeps AKBai running reliably, scales with the user base, and keeps Anton sane as a solo founder. These are distinct codebases, tools, and workflows that must be planned separately to avoid scope creep in the product build.

Four Operations Domains

Domain	What It Is
Admin Dashboard	Internal tool for Anton to manage users, subscriptions, flags, and observe system health
Billing Pipeline	Xendit webhook handling, subscription state machine, grace periods, plan enforcement
Support Infrastructure	WhatsApp Business API setup, response templates, triage and escalation workflows
Monitoring & Alerting Stack	Sentry, UptimeRobot, PostHog, health check endpoints for all 4 external APIs

Ops vs Product — Key Principle

The product roadmap builds what users pay for. The ops roadmap builds what lets Anton sleep at night. Both are required. Neither is optional.

OPS Build 0 — Monitoring & Alerting Foundation

OPS BUILD 0 | Must ship before any beta user onboards | ~8 hours | Phase 0A
D97706

Why This Ships First

Without monitoring you are blind. If the Claude API goes down during a BIR deadline, you will not know until a user WhatsApps you in a panic. This build creates visibility into every layer of the stack before a single real user touches the product.

Tools & Setup

Tool	Purpose	Cost	Setup Priority
Sentry	JavaScript error tracking, crash reports, source maps	\$0–\$26/mo	CRITICAL — Day 1
UptimeRobot	HTTP uptime monitoring for all API endpoints + PWA	\$7/mo Pro	CRITICAL — Day 1
PostHog	Product analytics, user funnels, Sense Check Gate signals	\$0 (free up to 1M events)	CRITICAL — Day 1
Supabase Dashboard	Built-in DB metrics, query performance, RLS audit	Included in Pro	CRITICAL — Day 1
Better Uptime (optional)	Public status page + on-call push alerts	\$0–\$20/mo	NICE TO HAVE

Health Check Endpoints

Create a /api/health endpoint in Next.js that pings all 4 external dependencies and returns a status JSON. UptimeRobot monitors this every 5 minutes. If any dependency fails, Anton gets a push notification within 5 minutes.

Dependency	Check Method	Failure Action
Supabase	SELECT 1 query	Show 'service degraded' banner in app
Claude API (Anthropic)	Lightweight ping with 10-token prompt	Fall back to static FAQ responses, show AI unavailable banner
Claude Haiku Vision	Receipt scan test (lightweight)	Disable receipt scanning, show manual entry prompt
Xendit	Webhook endpoint health	Pause subscription billing, log for manual retry

Anton's Alert Protocol

1. UptimeRobot detects downtime → Push notification to Anton's phone within 5 minutes
2. Anton checks /api/health endpoint to identify which dependency failed
3. Check dependency status page (status.anthropic.com, status.supabase.com, etc.)
4. Post in-app banner: 'Ang AKBai ay may technical issue ngayon. Bumabalik kami agad.' (Taglish)
5. Resolve or escalate per OPS Flow 2 Incident Runbook (see Operations Playbook)

Effort Estimate

Task	Hours
Sentry integration + source maps	2h
UptimeRobot setup + /api/health endpoint	2h
PostHog integration + event taxonomy	3h
Alert protocol documentation	1h
Total	8 hours

OPS Build 1 — Billing Pipeline

OPS BUILD 1 | Xendit integration + subscription state machine | ~14 hours |
Phase 1 — Build 4

D97706

Why This Is Separate from the Product Build

The product build handles what users see when they upgrade (the UI, the paywall, the upgrade CTA). The billing pipeline handles what happens behind the scenes: webhook receipt, idempotency checks, subscription state transitions, grace periods, and plan enforcement at the API layer. Getting this wrong means double-charging users or failing to cut off expired subscriptions — both are catastrophic for a solo founder's reputation.

Subscription State Machine

State	Trigger	What Happens
FREE	New signup	Free tier active, 10 queries/day limit enforced (Haiku-only routing)
ACTIVE	Xendit payment.succeeded webhook	Pro/Business features unlocked, limit removed
GRACE	Xendit payment.failed webhook	Pro features retained for 3 days, daily push reminder sent
EXPIRED	Grace period ends (72h cron)	Downgraded to Free tier, data preserved, upgrade prompt shown
CANCELLED	User cancels	Active until end of billing period, then FREE
REFUNDED	Xendit refund processed	Immediate downgrade to TRIAL

Webhook Idempotency (Critical)

Xendit webhooks can fire multiple times for the same event (network retry on timeout). The handler MUST deduplicate by `payment_id` before processing any state transition. Use a `webhook_events` table in Supabase with a unique constraint on (`payment_id`, `event_type`). If the row already exists, return 200 OK immediately without processing.

Grace Period Cron

A Supabase Edge Function runs every 6 hours. It queries all subscriptions in GRACE state where `grace_started_at` is older than 72 hours. For each expired subscription: set state to EXPIRED, send final push notification, enforce Free tier limits immediately.

Plan Enforcement Architecture

Plan enforcement happens at the API middleware layer, not in the UI. Every AI query checks the user's current subscription state from Supabase before processing. This prevents tier bypass via direct API calls. The check adds ~5ms latency.

Effort Estimate

Task	Hours
Xendit webhook handler + idempotency table	3h
Subscription state machine (Supabase schema + transitions)	3h
Grace period cron (Edge Function)	2h
Plan enforcement middleware	2h
Xendit test mode integration testing	2h
Billing event logging + admin visibility	2h
Total	14 hours

OPS Build 2 — Admin Dashboard

OPS BUILD 2 | Internal visibility + control panel for Anton | ~18 hours | Phase 1 — Post-launch (Month 2)

D97706

Philosophy

The admin dashboard is not a product feature. It is Anton's cockpit. For 0–200 users it can be a combination of Supabase Studio + a simple Retool view. Do not over-engineer this — the goal is visibility and control, not beauty.

Dashboard Modules

Module	What Anton Sees	Build Approach	Status
User Overview	Total users, active today/week/month, new signups, churn	Supabase view + PostHog dashboard	Phase 1
MRR & Revenue	MRR breakdown by plan (Beta/Free/Pro/Business), MRR growth, Xendit reconciliation. Reference: Financial Model v2.	Supabase query + Xendit dashboard	Phase 1
Feature Usage	Most-used features, Kilala Kita completion rate, bootstrap path breakdown	PostHog funnels	Phase 1
Flag-as-Wrong Queue	All AI responses flagged by users, raw content, response correction needed	Custom Supabase table + Retool view	Phase 1
Subscription States	Count by state (TRIAL/ACTIVE/GRACE/EXPIRED), at-risk accounts	Supabase view	Phase 1
API Health	Current status of all 4 dependencies, last check time	/api/health endpoint	Phase 0A

Feature Flags via Supabase

Store feature flags in a Supabase config table (key, value, description, updated_at). The product reads these on each session. Anton can toggle features on/off for all users or specific user groups without a code deploy. Example flags: bir_features_enabled, voice_input_enabled, google_sheets_beta.

Flag	Default	Purpose
bir_features_enabled	true	Master switch for all BIR features
data_bootstrap_csv	true	Enable CSV/XLSX upload path in Kilala Kita
voice_input_beta	false	Enable Whisper voice input for beta users
google_sheets_oauth	false	Enable Google Sheets OAuth (Phase 2)
maintenance_mode	false	Show maintenance banner to all users

Effort Estimate

Task	Hours
Supabase views for user + revenue metrics	4h
PostHog dashboard configuration	2h
Feature flags config table + product integration	4h
Flag-as-Wrong queue + Retool view	4h
Subscription state overview query	2h
Basic admin auth (separate Supabase role)	2h
Total	18 hours

OPS Build 3 — Support Infrastructure

OPS BUILD 3 | WhatsApp Business API + triage system | ~10 hours | Phase 1
— 2 weeks before launch

0B7A6E

Support Channel Decision

Support runs through WhatsApp Business (primary) and Viber (secondary). Both are widely used in the Philippine MSME market. Email support is available but deprioritized — most target users prefer messaging over email. A formal ticketing system (Zendesk, Freshdesk) is overkill for 0–200 users; a structured WhatsApp workflow with a Notion triage log is sufficient.

WhatsApp Business Setup

Item	What To Do	Priority
Business account	Register AKBai WhatsApp Business account (Meta Business Suite)	CRITICAL
Display name	'AKBai Support' with AKBai logo	CRITICAL
Business hours	Set: Mon–Fri 8AM–8PM PHT (Solo founder hours)	CRITICAL
Auto-reply	Set welcome message + expected response time in Taglish	CRITICAL
Quick replies	Create 10 template responses for most common issues	IMPORTANT

Response Templates (Taglish)

Scenario	Template
General greeting	'Kamusta! Ito si AKBai Support. Paano kita matutulungan ngayon?'
Acknowledge receipt	'Salamat sa iyong mensahe! Nakuha na namin ito. Susugurin namin sa loob ng 4 na oras.'
Bug report ack	'Naiintindihan namin ang issue mo. Nag-log na kami nito at inaayos na. Aabisuhan ka namin pagkatapos.'
Feature request	'Salamat sa suggestion! Idinagdag na namin ito sa aming feature backlog.'
BIR question	'Para sa specific tax advice, kumonsulta sa isang CPA. AKBai ay nagbibigay ng general guidance lamang.'
Resolved	'Naayos na ang iyong issue. Pakitingnan at i-update mo kami kung may iba pang nararamdaman.'

Triage Protocol

Priority	Definition	Response Target	Example
P0 — Critical	App down, data loss, payment issue	1 hour (any time)	'AKBai won't load'
P1 — High	Feature broken, user cannot complete core task	4 hours (biz hours)	'BIR deadline not showing'
P2 — Medium	Unexpected behavior, confusion, minor bug	24 hours (biz hours)	'Button alignment weird'
P3 — Low	Feature request, general feedback, how-to question	48 hours (biz hours)	'Can you add X feature?'

Effort Estimate

Task	Hours
WhatsApp Business account setup + verification	2h
Auto-reply + template messages setup	2h
Notion triage log template creation	1h
Response template library (10 templates)	2h
Viber Business account setup	1h
Support SLA documentation for website/app	2h
Total	10 hours

OPS Build 4 — BIR Calendar Management

OPS BUILD 4 | BIR deadline data pipeline | ~6 hours | Phase 1 — Build 3 dependency

7D3C98

Why BIR Calendar Is an Operations Problem

BIR deadlines change every year. The TRAIN Law and subsequent revenue regulations regularly adjust filing dates, add new deadlines, and modify form requirements. If the `bir_deadlines` table in Supabase has wrong dates, AKBai sends wrong alerts — which is legally and reputationally damaging. BIR calendar management must be treated as an ongoing operational responsibility, not a one-time data entry task.

Calendar Data Architecture

Column	Type	Description	Example
<code>id</code>	UUID	Primary key	<code>uuid</code>
<code>deadline_type</code>	TEXT	Type of BIR filing	<code>quarterly_income_tax</code>
<code>form_number</code>	TEXT	BIR form reference	<code>1701Q</code>
<code>due_date</code>	DATE	Filing/payment deadline	<code>2026-05-15</code>
<code>bir_type_applicable</code>	TEXT[]	Array: 8pct, graduated, or both	<code>{8pct, graduated}</code>
<code>notes</code>	TEXT	Any special conditions	<code>Extended due to Eid al-Fitr</code>
<code>updated_at</code>	TIMESTAMP	Last admin update	<code>2026-01-15 09:00:00</code>
<code>source_url</code>	TEXT	BIR RMC/circular ref	<code>https://bir.gov.ph/...</code>

Annual Calendar Refresh Workflow

- January: Monitor BIR website and official gazette for new Revenue Memorandum Circulars (RMCs)
- Cross-reference with previous year's calendar — identify changes
- Update `bir_deadlines` table in Supabase admin UI (no code deploy needed)
- Run a query to verify all deadlines have `source_url` populated
- Test that push notifications fire correctly for the updated dates in staging
- Confirm at least 2 weeks before first Q1 deadline of the year

BIR Deadline Categories to Track

Category	Key Deadlines
Percentage Tax (Non-VAT)	Monthly: Form 2551M (20th of following month)
Income Tax (8% flat)	Quarterly: Form 1701Q (May 15, Aug 15, Nov 15); Annual: Form 1701A (Apr 15)
Income Tax (Graduated)	Quarterly: Form 1701Q same dates; Annual: Form 1701 (Apr 15)
VAT	Monthly: Form 2550M (20th); Quarterly: Form 2550Q (25th after quarter)
Withholding Tax	Monthly: Form 0619-E / 1601-EQ
Registration	Annual: Form 0605 (Jan 31)

Effort Estimate

Task	Hours
bir_deadlines table schema + RLS policies	2h
Admin update UI (simple Supabase form)	2h
Annual refresh workflow documentation	1h
First year calendar data entry (2026)	1h
Total	6 hours

Infrastructure Cost-Trigger Milestones (not calendar-based):

Hosting (Vercel → Cloudflare Pages): M1–M6 ₱0 → Month 7 ₱286 (\$5)

Database (Supabase free → Pro): M1–M9 ₱0 → Month 10 ₱1,430 (\$25)

WhatsApp Business API: M1–M6 ₱0 → Month 7 ₱572 (\$10)

Monitoring (PostHog/Sentry free): All Y1–Y3 ₱0

Operations Roadmap — Timeline

OPS Build	Name	Phase	Effort	Dependency
OPS Build 0	Monitoring & Alerting Foundation	Phase 0A — Week 1-2	8h	None — ships first
OPS Build 1	Billing Pipeline	Phase 1 — Weeks 6-7	14h	Product Build 4
OPS Build 2	Admin Dashboard	Phase 1 — Weeks 9-10	18h	OPS Build 1
OPS Build 3	Support Infrastructure	Phase 1 — 2 wks before	10h	None (parallel)
OPS Build 4	BIR Calendar Mgmt	Phase 1 — Week 5	6h	Product Build 3
TOTAL	All Ops Builds	Phase 0A–Phase 1	56h	—

Ops Build Dependencies on Product Builds

Ops Build	Depends On	Why
OPS Build 1 (Billing)	Product Build 4	Cannot build billing pipeline without subscription UI/schema in product
OPS Build 2 (Admin)	OPS Build 1 + PostHog	Admin dashboard is meaningless without billing data and analytics
OPS Build 4 (BIR Calendar)	Product Build 3	BIR deadline table must align with the BIR features schema

This is a hard gate before Phase 1 public launch. UAT testers are not charged — they receive free access in exchange for structured feedback on core flows (Kilala Kita, Receipt Scanner, Dashboard, Morning Briefing).

UAT Cost: ₱800 total (Claude Haiku API for 15 testers × 4 weeks)

Monitoring: PostHog free only

WhatsApp Business API: NOT active

Payments: NOT active (no Xendit, no billing)

Analytics: PostHog free tier

Email: Resend free tier

Hosting: Cloudflare Pages free tier

Database: Supabase free tier

Claude API: Haiku only (no Sonnet) — testers can use the AI assistant

UAT Stack (stripped-down):

Before any public-facing launch, a User Acceptance Testing (UAT) environment runs for 2–4 weeks with 15–20 invited testers.

OPS Build UAT — Pre-Launch Testing Environment

Phase 2+ Operations Roadmap

After the Sense Check Gate passes (target: after Month 12, when AKBai reaches ~400 users and ₱63,773 cumulative net profit per Y1 projections), operations infrastructure should be extended to support growth. Key Phase 2 ops items include:

- OPS Build 5: Automated user onboarding email sequences (PostHog + SendGrid/Resend)
- OPS Build 6: Referral tracking pipeline (referral_codes table + conversion attribution)
- OPS Build 7: Advanced admin dashboard with cohort analysis and churn prediction signals
- OPS Build 8: Multi-agent support bot (AI-assisted first-response to common support tickets)

Ops Roadmap Summary

Metric	Value
Total ops build effort (Phase 0A → Phase 1 launch)	56 hours
Runs parallel to product builds?	Yes — ops are separate, can work concurrently
Solo founder feasibility	Yes — each build is self-contained 1–3 day sprints
Phase 2 ops additions planned	4 more builds identified (OPS Builds 5–8)
Companion documents	AKBai_Complete_Roadmap_v8.pdf (product), AKBai_Operations_Playbook.pdf (process flows)

This Operations Roadmap is a DRAFT for Anton's review. Once confirmed, the ops builds will be incorporated into the main AKBai Complete Roadmap as a dedicated Ops Roadmap section alongside the product phases.